Learn generative AI · zero to expert

GenAI Cheat Sheets

17 subject cheat sheets for fast revision and interview preparation.

created by

Harsha Varthan E P

Cheat sheet

Python

Syntax, data structures, OOP, idioms and interview gotchas

Basics

Types	`int`, `float`, `str`, `bool`, `None`, `list`, `tuple`, `dict`, `set`
Mutable	list, dict, set (and most objects). Immutable: int, float, str, tuple, frozenset
Truthy / falsy	Falsy: `0`, `""`, `[]`, `{}`, `None`, `False`. Everything else is truthy
Identity vs equality	`is` compares objects, `==` compares values. Use `is None`
f-string	`f"{name!r} {price:.2f} {pct:.1%}"`
Slicing	`s[start:stop:step]`, `s[`
Unpacking	`a, *rest = [1, 2, 3]`, `x, y = y, x` swaps

Data structures

python

```python
nums = [3, 1, 2]; nums.append(4); nums.sort(); nums.pop()       # list
d = {"a": 1}; d.get("b", 0); d.setdefault("c", []); d.items()   # dict
s = {1, 2} | {2, 3}; s & {2}; s - {1}                           # set ops
from collections import Counter, defaultdict, deque
Counter("banana").most_common(1)      # [('a', 3)]
g = defaultdict(list); g["k"].append(1)
q = deque([1, 2]); q.appendleft(0); q.popleft()
import heapq; heapq.nsmallest(2, [5, 1, 4])
```

Comprehensions and functional

List	`[x*x for x in xs if x > 0]`
Dict	`{k: v for k, v in pairs}`
Generator	`sum(x*x for x in xs)` (lazy, memory-friendly)

Sorting	`sorted(items, key=lambda r: (-r["score"], r["name"]))`
zip / enumerate	`for i, (a, b) in enumerate(zip(xs, ys))`
any / all	`any(x < 0 for x in xs)`

Functions

Defaults	Never use mutable defaults: `def f(x, acc=None): acc = acc or []`
***args / **kwargs**	Variable positional / keyword arguments
Keyword-only	`def f(a, *, verbose=False)`
Closures	Inner function remembers outer variables (`nonlocal` to modify)
Decorator	`@functools.wraps` keeps name and docstring
Type hints	`def f(x: list[int]) → dict[str, int]:`

OOP

```python
from dataclasses import dataclass
class Animal:
    count = 0                        # class attribute
    def __init__(self, name): self.name = name; Animal.count += 1
    def speak(self): raise NotImplementedError
    def __repr__(self): return f"Animal({self.name!r})"
class Dog(Animal):
    def speak(self): return "Woof"
@dataclass(frozen=True)
class Point: x: float; y: float
```

Dunder methods	`__init__` , `__repr__` , `__eq__` , `__len__` , `__iter__` , `__enter__/__exit__`
@staticmethod vs @classmethod	No self / receives the class (`cls`), used for alternative constructors
@property	Method accessed like an attribute, with validation

MRO	Method resolution order for multiple inheritance (`Class.__mro__`)

Errors, files, JSON

```python
try:
    data = json.loads(text)
except json.JSONDecodeError as e:
    log.warning("bad json: %s", e)
else:
    process(data)
finally:
    cleanup()
with open("f.txt", encoding="utf-8") as f:
    for line in f: ...
```

Concurrency

GIL	One thread runs Python bytecode at a time in standard CPython
threading	I/O-bound work (API calls, downloads)
multiprocessing	CPU-bound work
asyncio	Many concurrent I/O tasks: `await asyncio.gather(*tasks)`

Complexity

list	index O(1), append O(1), insert / search / remove O(n)
dict / set	get, set, in: O(1) average
sort	O(n log n) (Timsort, stable)

> ⚠ **Watch out**
>
> Classic traps: mutable default arguments, modifying a list while iterating, `is` vs `==`, integer division `//`, late binding in lambdas inside loops, shallow copies of nested lists.

Cheat sheet

SQL

Queries, joins, windows, performance and the classic interview problems

Query skeleton and order

```sql
SELECT col, AGG(x) AS a        -- 5
FROM t JOIN u ON ...           -- 1
WHERE row_filter               -- 2
GROUP BY col                   -- 3
HAVING group_filter            -- 4
ORDER BY a DESC                -- 6
LIMIT 10;                      -- 7
```

Filtering

Comparison	`=`, `<>`, `>`, `BETWEEN a AND b`, `IN (...)`, `LIKE 'A%'`, `ILIKE` (Postgres)
NULL	`IS NULL`, `IS NOT NULL`, `COALESCE(x, 0)`, `NULLIF(a, b)`
Conditional	`CASE WHEN x > 0 THEN 'pos' ELSE 'neg' END`
Dedupe	`SELECT DISTINCT`, `COUNT(DISTINCT col)`

Joins

INNER	Matches only
LEFT	All left rows; anti-join with `WHERE right.id IS NULL`
FULL OUTER	All rows both sides
CROSS	Cartesian product
SELF	`employees e JOIN employees m ON e.manager_id = m.id`
Semi-join	`WHERE EXISTS (SELECT 1 FROM ...)`

Window functions

```sql
ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC)
RANK() / DENSE_RANK() OVER (ORDER BY score DESC)
SUM(x) OVER (ORDER BY d ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
AVG(x) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)   -- 7-day moving
average
LAG(x, 1) OVER (ORDER BY d), LEAD(x) OVER (...), NTILE(4) OVER (...)
FIRST_VALUE(x) OVER (PARTITION BY g ORDER BY d)
```

Classic problems

```sql
-- Nth highest
WITH r AS (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rk FROM emp)
SELECT DISTINCT salary FROM r WHERE rk = 3;
-- Top 2 per department
SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) rn
FROM emp) t
WHERE rn ⩽ 2;
-- Duplicates
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
-- Month-on-month growth
WITH m AS (SELECT DATE_TRUNC('month', d) mon, SUM(amt) rev FROM sales GROUP BY 1)
SELECT mon, rev, ROUND(100.0 * (rev - LAG(rev) OVER (ORDER BY mon)) / LAG(rev) OVER
(ORDER BY mon), 1) pct FROM m;
-- Streaks (consecutive login days)
SELECT user_id, MIN(d), MAX(d), COUNT(*) days FROM
 (SELECT user_id, d, d - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY d)) * INTERVAL
'1 day' grp FROM logins) t
GROUP BY user_id, grp HAVING COUNT(*) ⩾ 3;
```

Design and performance

Keys	Primary key (unique, not null), foreign key (referential integrity), unique, check, default
Normal forms	1NF atomic values; 2NF no partial dependency; 3NF no transitive dependency
Indexes	B-tree for equality/range; composite index follows leftmost-prefix rule
EXPLAIN	`EXPLAIN ANALYZE` shows the plan: look for sequential scans on big tables

ACID	Atomicity, Consistency, Isolation, Durability
Isolation levels	Read uncommitted < read committed < repeatable read < serializable
OLTP vs OLAP	Many small transactions vs large analytical scans (warehouses, columnar)
Star schema	Fact table (events, measures) + dimension tables (who, what, when, where)
pgvector	`ORDER BY embedding ⟺ query_vec LIMIT k` (cosine distance)

⚠ **Watch out**

Traps: `= NULL` never matches; `COUNT(col)` skips NULLs but `COUNT(*)` doesn't; integer division in some engines; aliases are not usable in WHERE; LEFT JOIN turns into INNER when you filter the right table in WHERE.

Cheat sheet

Maths & Statistics

Linear algebra, probability, statistics and calculus for AI

Linear algebra

Vector / matrix / tensor	1-D list / 2-D grid / n-D array of numbers
Dot product	$a \cdot b = \Sigma\, a_i b_i$; large when vectors align
Cosine similarity	$a \cdot b \,/\, (\|a\|\|b\|)$ in [−1, 1]
Matrix multiply	(m×n)(n×p) → (m×p); each layer of a network is $XW + b$
Eigenvectors / PCA	Directions of maximum variance
Norms	L1 = $\Sigma\|x\|$ (sparsity), L2 = $\sqrt{\Sigma x^2}$ (regularisation)

Probability

Conditional	$P(A\|B) = P(A \cap B) \,/\, P(B)$
Bayes	$P(A\|B) = P(B\|A)\, P(A) \,/\, P(B)$
Independence	$P(A \cap B) = P(A)\, P(B)$
Expected value	$E[X] = \Sigma\, x\, P(x)$
Distributions	Bernoulli, binomial, Poisson (counts), normal, uniform, exponential
Softmax	$\exp(z_i) \,/\, \Sigma \exp(z_j)$; temperature divides z before softmax

Statistics

Central tendency	Mean (sensitive to outliers), median (robust), mode
Spread	Variance, standard deviation, IQR
CLT	Sample means are approximately normal for large n
Hypothesis test	H_o vs H_1; p-value < α (e.g. 0.05) → reject H_o

Errors	Type I = false positive (α); Type II = false negative (β); power = 1 − β
Confidence interval	Range likely to contain the true parameter at a given confidence
Tests	t-test (means), chi-square (categories), ANOVA (3+ groups)
Correlation	Pearson (linear), Spearman (rank); correlation ≠ causation

Calculus and optimisation

Derivative	Rate of change / slope
Chain rule	`d/dx f(g(x)) = f'(g(x)) g'(x)`, the core of backprop
Gradient descent	`θ ← θ - η ∇L(θ)`
Convex	One global minimum; neural nets are non-convex

Information theory

Entropy	Uncertainty: `-Σ p log p`
Cross-entropy	Loss for classification / next-token prediction
KL divergence	How one distribution differs from another (used in RLHF, distillation)
Perplexity	`exp(cross-entropy)`; lower = better language model

Cheat sheet

Pandas & NumPy

Data wrangling commands you will use every day

NumPy

```python
import numpy as np
a = np.arange(12).reshape(3, 4); a.shape, a.T, a[:, 1], a[a > 5]
np.mean(a, axis=0); np.where(a > 5, 1, 0); np.concatenate([a, a]); a @ a.T
```

Broadcasting	Arrays of different shapes combine when trailing dimensions match or are 1

Pandas: load and inspect

```python
df = pd.read_csv("f.csv", parse_dates=["date"])
df.head(); df.info(); df.describe(include="all"); df.shape; df.dtypes; df.nunique()
```

Select and filter

Columns	`df["a"]`, `df[["a", "b"]]`
Rows by label / position	`df.loc[rows, cols]`, `df.iloc[0:5, 1:3]`
Boolean filter	`df[(df.a > 1) & (df.b == "x")]`, `df.query("a > 1 and b == 'x'")`
Membership	`df[df.city.isin(["Chennai", "Pune"])]`

Clean

Missing	`isna().sum()`, `fillna(v)`, `dropna(subset=[...])`, `interpolate()`
Duplicates	`duplicated()`, `drop_duplicates(subset=[...], keep="last")`
Types	`astype("int")`, `pd.to_datetime()`, `pd.to_numeric(errors="coerce")`

Strings	`.str.lower()`, `.str.strip()`, `.str.contains("x")`, `.str.extract(r"(\d+)")`
Rename / drop	`rename(columns={...})`, `drop(columns=[...])`

Transform and aggregate

python

```python
df.groupby("city").agg(orders=("id", "count"), revenue=("amt", "sum"))
df.pivot_table(index="city", columns="month", values="amt", aggfunc="sum", fill_value=0)
df.merge(other, on="id", how="left")      # inner / left / right / outer
pd.concat([df1, df2], ignore_index=True)
df["rank"] = df.groupby("city")["amt"].rank(ascending=False)
df["roll7"] = df["amt"].rolling(7).mean()
df.melt(id_vars="id"); df.explode("tags"); df.sample(frac=0.1, random_state=1)
```

> ⚠ **Watch out**
>
> Avoid row loops; use vectorised operations. Use `.copy()` after filtering to avoid `SettingWithCopyWarning`.

Cheat sheet

Machine Learning

Algorithms, evaluation, workflow and interview essentials

Problem types

Regression	Predict a number (price, demand)
Classification	Predict a category (spam, churn)
Clustering	Group unlabeled data (segments)
Ranking / recommendation	Order items for a user
Anomaly detection	Find rare unusual events (fraud)

Algorithms

Linear regression	Minimise squared error; assumes linearity
Logistic regression	Sigmoid of linear score; log-loss
k-NN	Vote of nearest neighbours; needs scaling
Naive Bayes	Bayes rule with independence assumption; fast text baseline
SVM	Max-margin boundary; kernels for non-linear
Decision tree	Recursive splits by Gini or entropy
Random forest	Bagging + random features
Gradient boosting	Sequential trees on residuals (XGBoost, LightGBM, CatBoost)
K-Means	Minimise within-cluster distance; choose K via elbow / silhouette
DBSCAN	Density clusters, finds noise
PCA / t-SNE / UMAP	Dimensionality reduction / visualisation

Evaluation

Splits	Train / validation / test; stratify for imbalance; time-based split for time series
Cross-validation	k-fold average for robust estimates
Classification	Accuracy, precision, recall, F1, ROC-AUC, PR-AUC, confusion matrix
Regression	MAE, MSE, RMSE, R^2, MAPE
Imbalance	Class weights, resampling (SMOTE), threshold tuning, PR-AUC
Calibration	Predicted probabilities match real frequencies

Bias, variance, regularisation

Underfit	High bias → more features, complex model
Overfit	High variance → more data, regularise, simplify
L1 (Lasso)	Sparse weights, feature selection
L2 (Ridge)	Small weights, stability
Early stopping	Stop when validation stops improving

Workflow

1 Define problem — 2 Data — 3 EDA — 4 Features — 5 Baseline —
6 Model — 7 Tune — 8 Evaluate — 9 Deploy — 10 Monitor drift

Feature scaling	StandardScaler (z-score), MinMaxScaler
Encoding	One-hot, ordinal, target encoding (beware leakage)
Leakage	Fit preprocessing on training data only; time-aware splits
Drift	Data drift (inputs change), concept drift (relationship changes)

Cheat sheet

Deep Learning

Neural networks, training, architectures and PyTorch

Building blocks

Neuron	`σ(w·x + b)`
Activations	ReLU, GELU, SwiGLU (transformers), sigmoid, tanh, softmax
Losses	MSE, binary cross-entropy, cross-entropy, contrastive (embeddings)
Backprop	Chain rule from loss to every weight
Optimisers	SGD (+momentum), Adam, AdamW (decoupled weight decay)
LR schedule	Warmup then cosine / linear decay

Training stability

Normalisation	BatchNorm (CNNs), LayerNorm / RMSNorm (transformers)
Residual connections	`x + f(x)` lets gradients flow in deep nets
Regularisation	Dropout, weight decay, augmentation, early stopping
Gradient clipping	Prevent exploding gradients
Mixed precision	fp16/bf16 for speed and memory

Architectures

MLP	Stacked fully-connected layers
CNN	Convolutions + pooling; images
RNN / LSTM / GRU	Sequences with recurrence; gates fight vanishing gradients
Transformer	Attention + feed-forward + residual + norm
Autoencoder / VAE	Compress and reconstruct; latent space

GAN	Generator vs discriminator
Diffusion	Learn to denoise; generate from noise

PyTorch loop

```python
model.train()
for xb, yb in loader:
    xb, yb = xb.to(device), yb.to(device)
    loss = loss_fn(model(xb), yb)
    opt.zero_grad(); loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step(); sched.step()
model.eval()
with torch.no_grad(): preds = model(x_val)
```

Cheat sheet

NLP

Classic NLP, tokenisation and embeddings

Pipeline

1 Text — 2 Normalise — 3 Tokenise —

4 Represent (BoW / TF-IDF / embeddings) — 5 Model — 6 Output

Concepts

Tokenisation	Word, subword (BPE, WordPiece, SentencePiece/Unigram), character
Stopwords	Very frequent words often removed in classic NLP
Stemming vs lemmatisation	Chop suffix vs dictionary base form
BoW / TF-IDF	Count vectors / weighted by rarity across documents
n-grams	Sequences of n tokens ("New York")
Word2Vec / GloVe / fastText	Static word embeddings (fastText handles sub-words)
Contextual embeddings	BERT-style: same word, different vector by context
Sentence embeddings	Sentence-Transformers, embedding APIs for search and RAG

Tasks

Classification	Sentiment, intent, topic, spam
NER	People, organisations, places, dates
QA	Extractive (span) vs generative
Summarisation	Extractive vs abstractive
Translation	Seq2seq / LLMs

Metrics

BLEU	n-gram overlap (translation)
ROUGE	Recall-oriented overlap (summaries)
BERTScore	Embedding similarity of outputs
Perplexity	Language-model quality

Pro tip

For modern generative tasks, n-gram metrics correlate poorly with quality; use rubric-based human or LLM-as-judge evaluation as well.

Cheat sheet

Transformers & LLMs

Architecture, training, inference and model selection

Architecture

Attention	$\text{softmax}(QK^T/\sqrt{d_k})\ V$
Multi-head	Several attention heads concatenated
Causal mask	Decoder tokens see only previous tokens
Positional info	Sinusoidal, learned, RoPE (rotary), ALiBi
Block	Attention → add & norm → feed-forward → add & norm
Families	Encoder (BERT), decoder (GPT, Claude, Llama, Gemini), encoder-decoder (T5)
MoE	Mixture-of-Experts routes each token to a few expert networks; more capacity per FLOP

Training

Pretraining	Next-token prediction on trillions of tokens
SFT	Supervised instruction tuning
Preference tuning	RLHF (reward model + PPO), DPO, RLAIF / constitutional methods
Reasoning training	RL on verifiable tasks (maths, code) to improve step-by-step thinking
Scaling laws	Loss falls predictably with parameters, data and compute

Inference

Decoding	Greedy, beam search, sampling (temperature, top-p, top-k)
KV cache	Stores past keys/values to avoid recomputation
Quantisation	8-bit / 4-bit weights (GPTQ, AWQ, GGUF)
Speculative decoding	Small draft model proposes tokens; big model verifies

Batching	Continuous batching (e.g. vLLM) for throughput
Context length	Long context vs retrieval trade-off; "lost in the middle" effect

Choosing a model

Quality	Task benchmarks + your own evaluation set
Cost / latency	Price per million tokens, time-to-first-token, tokens per second
Privacy	API vs self-hosted open weights
Features	Tool calling, structured output, vision, long context, fine-tuning support
Licence	Check open-weight licences for commercial use

Cheat sheet

Prompt Engineering

Patterns, templates and evaluation

Structure

Role	Expertise and perspective
Task	Clear action verb and goal
Context	Audience, background, data in tags
Format	Length, sections, JSON schema
Rules	Must / must not, tone, fallback behaviour

Techniques

Zero-shot	Instructions only
Few-shot	2–5 varied examples
Chain-of-thought	Reason step by step; final answer on its own line
Self-consistency	Sample several, majority vote
Tree-of-thoughts	Explore and evaluate several paths
ReAct	Interleave reasoning and tool actions
Prompt chaining	Pipeline of small prompts
Meta-prompting	Ask the model to improve your prompt
Prefill	Start the assistant's answer (e.g. `{`) to force format

Templates

prompt

```
<role>You are a {expert}.</role>
<task>{task}</task>
<context>{data}</context>
<format>Return JSON: {"answer": str, "confidence": "low|medium|high", "sources": [str]}
</format>
<rules>Use only the context. If unsure, answer "unknown".</rules>
```

Evaluation

Test set	Typical, edge, adversarial inputs
Checks	Schema validity, length, required keywords, banned content
LLM-as-judge	Rubric with 1–5 scores and reasons; calibrate against humans
Versioning	Prompts in Git; log version per response

Cheat sheet

LLM APIs

Messages, streaming, tools, structured output and reliability

Concepts

Roles	system (rules), user (input), assistant (model turns), tool (results)
Stateless	Resend history each call
Tokens	Billed input + output; check `usage` in responses
Streaming	Server-sent events; show tokens as they arrive
Tool calling	Model returns tool name + JSON args; you execute and return result
Structured output	JSON schema mode; validate with Pydantic
Vision / files	Send images or PDFs as content blocks
Embeddings endpoint	Text → vector for search

Free ways to practise

Google AI Studio	Free, rate-limited Gemini API key; free-tier data may be used to improve Google products
Ollama	Run open models locally, OpenAI-compatible endpoint
Hugging Face	Free models, Spaces and limited free inference
Transformers.js	Run small models in the browser (used in this app's RAG Lab)

Reliability

Errors	400 bad request, 401 auth, 429 rate limit, 5xx server
Retries	Exponential backoff with jitter; idempotent requests
Timeouts	Always set; fall back to a smaller model
Secrets	Environment variables / secret managers; never in client-side code for production

Cost control	Smaller models, caching, max tokens, batch APIs, prompt compression

Cheat sheet

Vectors & RAG

Embeddings, vector databases, retrieval and evaluation

Pipeline

1 Ingest — 2 Parse — 3 Chunk — 4 Embed — 5 Index — 6 Retrieve —
7 Rerank — 8 Generate — 9 Evaluate

Design choices

Chunking	Fixed, recursive, structure-aware, semantic; 300–800 tokens, 10–20% overlap
Embedding model	Quality on your language/domain, dimensions, cost, speed
Index	HNSW (fast, memory-heavy), IVF (clustered), flat (exact, small data)
Distance	Cosine, dot product, Euclidean (normalise vectors for cosine)
Hybrid	BM25 + dense; fuse with reciprocal rank fusion
Reranker	Cross-encoder over top-N candidates
Metadata	Source, page, section, date, tenant, ACL; filter before ranking
Context assembly	Deduplicate, order by relevance, cite sources, respect token budget

Advanced

Query rewriting	Clarify / expand the question
Multi-query / RAG-Fusion	Several queries, merged results
HyDE	Embed a hypothetical answer
Parent-child	Retrieve small, return larger parent
Contextual retrieval	Prepend chunk-specific context before embedding

GraphRAG	Entity graph + community summaries
Agentic RAG	Agent decides searches iteratively

Evaluation

Retrieval	Recall@k, precision@k, MRR, nDCG, hit rate
Generation	Faithfulness, answer relevance, context precision/recall, citation accuracy
Tools	RAGAS, DeepEval, TruLens, Arize Phoenix (open source)

⚠ **Watch out**

Common failures: bad PDF parsing, chunks without context, stale indexes, missing permissions filtering, too many chunks (noise), no abstain path.

Cheat sheet

Agents & MCP

Loops, patterns, tools, memory, protocols and safety

Core

Agent	LLM + tools + memory + control loop + stop condition
ReAct	Thought → Action → Observation loop
Planning	Decompose goal; plan-and-execute; re-plan on failure
Reflection	Critique own output and retry
Memory	Short-term (context), long-term (DB / vectors), episodic (past runs)

Workflow patterns

Prompt chaining	Fixed sequence of steps
Routing	Classify and dispatch to specialist
Parallelisation	Sectioning or voting in parallel
Orchestrator-workers	Lead plans, workers execute subtasks
Evaluator-optimiser	Generate → critique → improve loop

MCP

Host / client / server	App / connector / tool provider
Primitives	Tools (actions), resources (data), prompts (templates)
Transports	stdio (local), streamable HTTP (remote)
Security	Trust only vetted servers; least privilege; review tool permissions

Safety and evaluation

Limits	Max steps, time, tokens, money
Approvals	Human-in-the-loop for irreversible actions
Sandboxing	Run code and browsers in isolated environments
Injection	Treat tool outputs and web content as untrusted
Metrics	Task success, steps, cost, latency, safety incidents, intervention rate

Cheat sheet

Fine-tuning

SFT, PEFT, LoRA, alignment and evaluation

When

Fine-tune for	Style, format, narrow skill, smaller/faster model, consistent behaviour
Don't fine-tune for	Fresh or frequently changing facts (use RAG)

Methods

SFT	Train on prompt → ideal response pairs
Full fine-tuning	Update all weights; expensive
PEFT	Update few parameters (LoRA, prefix tuning, adapters)
LoRA	Low-rank matrices A·B added to weight updates; rank r, alpha, target modules
QLoRA	LoRA on 4-bit (NF4) quantised base model
Distillation	Student mimics teacher outputs
Preference	RLHF (PPO with reward model), DPO, ORPO, KTO

Data

Format	JSONL chat messages; consistent system prompts
Size	Hundreds to thousands of high-quality varied examples
Split	Hold out a test set; deduplicate; remove PII

Hyperparameters

Learning rate	~1e-4 to 2e-4 for LoRA (lower for full fine-tunes)

Epochs	1–3; watch validation loss
Rank r	8–64; higher = more capacity

Risks

Overfitting	Memorises examples; poor generalisation
Catastrophic forgetting	Loses general abilities
Safety drift	Fine-tuning can weaken refusals; re-run safety evals

Pro tip

Free practice: Google Colab or Kaggle notebooks (free GPU quotas) with Hugging Face TRL / PEFT or Unsloth on a 0.5B–3B open model.

Cheat sheet

LLMOps & Deployment

Ship, evaluate, monitor and optimise

Stack

UI	Streamlit, Gradio, static HTML on GitHub Pages
Backend	FastAPI, serverless functions
Serving open models	vLLM, TGI, Ollama, llama.cpp
Packaging	Docker; pin dependencies (`requirements.txt`)
Free hosting	Hugging Face Spaces, Streamlit Community Cloud, GitHub Pages (static), Render free tier

Evaluation

Offline	Golden set, regression tests, LLM-as-judge, human review
Online	A/B tests, thumbs up/down, task completion, escalation rate
Gates	Block deploys if scores drop below thresholds

Observability

Traces	Prompt, context, tool calls, outputs, tokens, latency per step
Tools	Langfuse, Arize Phoenix, OpenTelemetry (open source options)
Alerts	Error spikes, latency p95, cost per day, safety flags

Optimisation

Cost	Model routing, caching, prompt compression, batching, shorter outputs
Latency	Streaming, smaller models, parallel retrieval, fewer chunks, regional deployment

Quality	Better retrieval, reranking, few-shot examples, fine-tuning

Versioning

Prompts	Stored in Git with IDs
Models	Pin exact model versions; test before upgrading
Data	Version indexes and datasets

Cheat sheet

Git, Linux & Docker

The developer toolkit every AI engineer uses

Git

terminal

```
git clone URL; git status; git add -A; git commit -m "msg"; git push
git switch -c feature/rag; git merge main; git rebase main
git log --oneline --graph; git diff; git stash; git restore file
git revert <sha>        # safe undo on shared branches
```

Branching	Feature branches + pull requests + code review
.gitignore	`.env`, `venv/`, `__pycache__/`, `*.pkl`, `node_modules/`
Merge vs rebase	Merge keeps history; rebase rewrites for a linear history (never on shared branches)

Linux

terminal

```
ls -la; cd; pwd; mkdir -p a/b; cp -r; mv; rm -r; cat; less; head; tail -f log.txt
grep -rn "error" .; find . -name "*.py"; chmod +x run.sh; ps aux; kill PID; top
curl -s URL | jq '.data'; export KEY=value; echo $PATH; ssh user@host
```

Docker

terminal

```
docker build -t app .; docker run -p 8000:8000 --env-file .env app
docker ps; docker logs -f ID; docker exec -it ID sh; docker compose up -d
```

Image vs container	Blueprint vs running instance
Layers and cache	Copy requirements and install before copying code

Cheat sheet

GenAI System Design

Framework, components and trade-offs for design rounds

Framework

1 Requirements — 2 Metrics — 3 Architecture — 4 Data and retrieval —
5 Model and prompts — 6 Evaluation and safety — 7 Scale, latency, cost

Clarifying questions

Users and scale	Who, how many, queries per second at peak
Latency	Time-to-first-token and total targets
Accuracy	Tolerance for errors; need for citations
Data	Sources, freshness, sensitivity, languages, permissions
Budget	Cost per query, infrastructure limits

Components

Gateway	Auth, rate limiting, quotas
Orchestrator	Prompting, tool calls, retries, routing
Retrieval	Ingestion pipeline, hybrid index, reranker, ACL filters
Models	Router between small and large; fallbacks
Guardrails	Input/output moderation, PII, injection defence
Caching	Exact and semantic caches; embedding cache
Observability	Traces, metrics, evaluation pipelines, feedback

Back-of-envelope

Tokens per query	Prompt + context + output (e.g. 3,000 in + 400 out)
Daily cost	Queries × tokens × price per token
Throughput	QPS × tokens per request ÷ tokens per second per replica

Pro tip

Always close with risks and how you would measure success after launch.

Cheat sheet

Responsible AI

Risks, controls and regulations

Risks

Hallucination	Fabricated facts or citations
Bias	Unequal treatment across groups
Privacy	PII exposure, memorisation, logging sensitive data
Security	Prompt injection, data exfiltration, jailbreaks, insecure tools
Misuse	Deepfakes, fraud, harmful content
IP	Copyright and licensing of data and outputs

Controls

Grounding	RAG with citations and abstention
Guardrails	Moderation, policy prompts, output validation
Privacy	Minimisation, masking, encryption, retention policies, access control
Security	Least privilege, sandboxing, approvals, red-teaming
Transparency	Disclose AI use; model cards; user controls
Governance	Risk registers, reviews, incident response

Frameworks and laws

EU AI Act	Risk-based obligations; transparency duties
India DPDP Act 2023	Consent and duties for personal data processing
GDPR	EU data protection rights and duties
NIST AI RMF	Govern, Map, Measure, Manage

ISO/IEC 42001	AI management system standard
OWASP Top 10 for LLM Apps	Common LLM security risks checklist